SQL Injection — Interview Prep Notes

---

1. Logical Description

- Untrusted input concatenated into SQL string instead of treated as data
- No code/data separation → input breaks out of literal context → parsed as SQL syntax
- Root: query built via string concat, not parameterized
- Same family as Command Injection/XXE — untrusted data reaching an interpreter

---

2. OWASP Category & CWE

- **OWASP 2021**: A03:2021 – Injection
- **CWE-89**: Improper Neutralization of Special Elements used in an SQL Command
- Related: **CWE-943** (Improper Neutralization of Special Elements in Data Query Logic), **CWE-20** (Improper Input Validation — parent)

---

3. Common Variants

- **In-band**: Error-based, Union-based
- **Blind**: Boolean-based, Time-based
- **Out-of-band (OOB)**: DNS/HTTP exfil via Collaborator/interactsh
- **First-order** vs **Second-order** (stored)
- **Stacked/Piggybacked queries** (`;` chained statements)
- Sub-context: Numeric vs String-based vs Search(LIKE)-based
- Related family: NoSQL Injection, ORM Injection (raw query bypass)

---

4. Attack Surface / Entry Points

- GET/POST params, JSON/XML body, path params
- Cookies, HTTP headers (UA, Referer, X-Forwarded-For, custom tenant/API-key headers)
- Login/password reset/OTP forms
- Search, sort/order-by, filter, pagination params
- Second-order: registration fields, file upload metadata, CSV/bulk import
- APIs (REST/GraphQL/SOAP), WebSockets, webhooks, chatbots
- Admin panels, internal tools (low test priority, high impact)

---

5. Testing Workflow (Phase → Burp / ZAP / Nuclei)

Phase	Verify	Burp	ZAP	Nuclei
Recon	All input points	Site Map/Spider	Spider+AJAX Spider	consumes gau/waybackurls/Katana list
Baseline	Normal response	Repeater baseline	Manual Editor+HUD	N/A
Syntax break	`'`, `"`, `--` breaks query	Repeater/Intruder	Active Scan SQLi rule	`-tags sqli`
Error-based	DB error leak	Repeater + Grep-Match	Alerts tab	`sqli-error-based` templates
Boolean blind	True/false diff	Repeater pairs + Comparer	Compare Requests	matcher on content/status
Time blind	`SLEEP()` / `WAIT FOR` delay	Repeater timing	Time-based sub-rule	`-it time-based`, duration matcher
Union-based	Column count, data extract	Repeater `ORDER BY` → `UNION SELECT`	Manual Editor	limited (stateful)
OOB	Callback confirm	Burp Collaborator	OAST/Interactsh	built-in interactsh, `-oob`
Exploitation	Full data/auth bypass	Repeater full chain	Manual	N/A (detection only)
WAF bypass	Encoding/case evasion	Intruder + encode rules	Fuzzer	`sqli-bypass-waf` templates

---

6. Evidence to Collect

- Full request/response pair (raw HTTP) with payload highlighted
- Baseline vs injected response diff (length/status/timing)
- DB error message / banner (version, engine type)
- Extracted data sample (redacted PII) proving read access
- Collaborator/interactsh interaction log (OOB proof)
- Screenshot of response timing (time-based) or boolean diff
- CVSS vector justification, affected endpoint/param list

---

7. Good PoC Structure

- Target endpoint + param + method
- Exact payload used
- Before/after response comparison
- Proof of impact: extracted DB version/table/data (not full dump — sample only)

- Repro steps (curl/Burp request) + screenshot
- Business impact one-liner (data class exposed)

---

8. Attack Chains

- SQLi → Auth Bypass → Account Takeover
- SQLi → Data Exfil → Credential Reuse → Lateral account compromise
- SQLi → Stacked Query → RCE (`xp_cmdshell`) → Full server compromise → Lateral movement
- Second-order SQLi → persistent backdoor via stored payload
- SQLi → Admin credential extraction → Admin panel access → further IDOR/BOLA chain

---

9. False Positives

- Legit apostrophes in names (O'Brien)
- Code/SQL snippets in text fields (forums, tutorials)
- Search terms literally "OR"/"AND"/"UNION" (product/band names)
- Authorized scanner traffic misclassified
- Naturally slow complex JOIN queries mistaken for time-based
- Pasted DB error text in support tickets (no real injection)

---

10. Prerequisites

- Injectable entry point, no parameterization
- No/weak input validation or WAF (or bypassable)
- Ability to observe response diff (blind) or external listener (OOB) + DB network egress capability
- Sufficient backend DB privileges to determine exploitation ceiling

---

11. Attacker Gains

- Full data exfiltration (credentials, PII, financial data)
- Auth bypass, privilege escalation
- Data tampering (INSERT/UPDATE/DELETE), integrity loss
- RCE via `xp_cmdshell` / `INTO OUTFILE` / `UTL_HTTP`
- Lateral movement, DoS (destructive stacked queries)

---

12. CIA Impact Table (with vector per variant)

Variant	C	I	A	Vector	Score
Error-based	H	N	N	`AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`	7.5
Union-based	H	N/L	N	same	7.5
Boolean blind	H	N	N	same	7.5
Time blind	H	N	L/N	same (A:L variant)	7.5
OOB	H	N	N	same (S:C if pivot)	7.5+
Stacked queries	H	H	H	`S:U/C:H/I:H/A:H`	9.8
SQLi→RCE	H	H	H	`S:C/C:H/I:H/A:H`	10.0
Auth bypass	H	M/H	N/L	`C:H/I:H/A:N`	9.1

---

13. Typical CVSS Severity/Vector

- Baseline: `AV:N/AC:L/PR:N/UI:N/S:U` — network, low complexity, pre-auth, no UI
- **High (7.5)**: read-only variants (error/union/blind/OOB)
- **Critical (9.1–10.0)**: stacked queries, auth bypass, RCE-chained (Scope Changed)

---

14. Business Impact — Financial App Context

- PCI-DSS: cardholder data exposure → Req 6.5.1 critical finding, fines
- Regulatory: GDPR Art.33, RBI/SOX reporting breach obligations
- Fraud risk: account takeover → unauthorized transactions
- Availability impact on transaction systems → SLA/financial loss
- Reputational damage, customer trust erosion, chargeback/legal liability
- Data integrity compromise → inaccurate financial records/audit failure

---

15. IOC / Signals

- SQL metachars in requests (`' OR 1=1--`, `UNION SELECT`, `SLEEP(`)
- Encoded/obfuscated payloads, scanner UA strings (sqlmap)
- Verbose DB error in response, timing anomalies matching sleep value
- Boolean differential (content-length/status diff)
- OOB callback hit, unexpected new DB objects, webshell file drop

---

16. Root Cause

- String concatenation/dynamic query building (code+data not separated)

- Missing parameterization/prepared statements
- ORM bypass via raw/native query methods
- Dynamic SQL inside stored procedures
- Over-privileged app DB account amplifies impact
- Inconsistent sanitization across codebase (legacy/admin endpoints)

---

17. Remediation / Secure Coding

- Parameterized queries/prepared statements (primary fix)
- ORM native query builder (avoid `.raw()`)
- Allowlist validation for non-parameterizable identifiers (table/column/order-by)
- Least privilege DB account (no DROP/FILE/EXEC unless needed)
- Centralized data access layer/shared wrapper library
- Escaping = last resort only, never primary control

---

18. Compensating Controls

- WAF (signature-based, bypassable, time-buy only)
- Database Activity Monitoring (DAM)
- Rate limiting/anomaly detection
- Network segmentation (no DB internet egress)
- Disable dangerous DB functions (`xp_cmdshell`, `LOAD_FILE`)
- Framework/CI lint rule blocking raw string-built queries

---

19. Repeated Occurrence → Security Posture Signal

- Indicates missing centralized DAL/shared query wrapper
- Secure coding training gap
- SAST not gating merges (WARNING-only, never promoted to BLOCK)
- Inconsistent code review rigor across teams/legacy code
- Systemic control failure (same framing as recurring IDOR) — compounding regulatory risk

---

20. Upstream Improvements

- Semgrep taint rule: source (request/header/cookie) → sink (raw execute) without parameterized sanitizer
- CodeQL dataflow query (`sql-injection` standard queries + custom sink for internal wrapper)
- Staged rollout: WARNING → ERROR/blocking (SQLi = high-confidence, fast-track to blocking vs IDOR)

- Framework-level default protections (ORM parameterization enforced by default)
- Design checklist: no raw query concat allowed; all DB access via shared library
- Linter rule flagging string concat/f-string into execute()

---

21. Sample Triage Response

> *Confirmed SQL Injection on `[endpoint]`, param `[param]`. Boolean-based blind confirmed via true/false differential (content-length delta observed). Time-based confirmed via `SLEEP(5)` — 5.2s response delay. No data extraction performed beyond DB version banner (confirmation only). CVSS 3.1: 7.5 (High) — `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`. Recommend immediate parameterization fix; escalate to Critical if stacked queries confirmed.*

---

22. Sample Developer Remediation Note

> *Root cause: query at `[file:line]` built via string concatenation of `username` param. Fix: replace with parameterized query using `[framework-specific bind method]`. Do not use string formatting/f-strings/concat for SQL. Refer to shared DB access wrapper `[lib name]` — use existing `.query(sql, params)` method. Retest with error/union/blind/OOB payloads post-fix before closure.*

---

23. Real-World Example

- **TalkTalk (2015)**: SQLi breach exposed ~157k customer records (names, bank details) — ICO fined £400k
- **Heartland Payment Systems (2008)**: SQLi-based breach → ~130M card numbers exposed, one of largest at the time
- Common bug bounty pattern: blind SQLi in search/filter param on internal admin tool → chained to full DB read via time-based extraction